```
        preprocessed_masks,      # Preprocessed masks
        image_names,             # Image names
        test_size=0.2,           # Proportion of data to be used for testing and va
        random_state=seed_value, # Seed value for reproducibility
    )

    # Split the remaining data into validation and test
    (
        images_validation,       # Validation images
        images_test,             # Test images
        masks_validation,        # Validation masks
        masks_test,              # Test masks
        names_validation,        # Validation image names
        names_test,              # Test image names
    ) = train_test_split(
        images_test_and_val,     # Combined test and validation images
        masks_test_and_val,      # Combined test and validation masks
        names_test_and_val,      # Combined test and validation image names
        test_size=0.5,           # Proportion of data to be used for testing
        random_state=seed_value, # Seed value for reproducibility
    )

# If SAVE_DATASETS is True, then save the data to the save_dir
if SAVE_DATASETS:
    save_datasets(
        training=(images_train, masks_train),             # Training images and mas
        test=(images_test, masks_test),                   # Test images and masks
        validation=(images_validation, masks_validation), # Validation images and m
        save_dir=save_dir,                                # Directory to save the d
    )
```

```
Data saved in Datasets/merged.
```

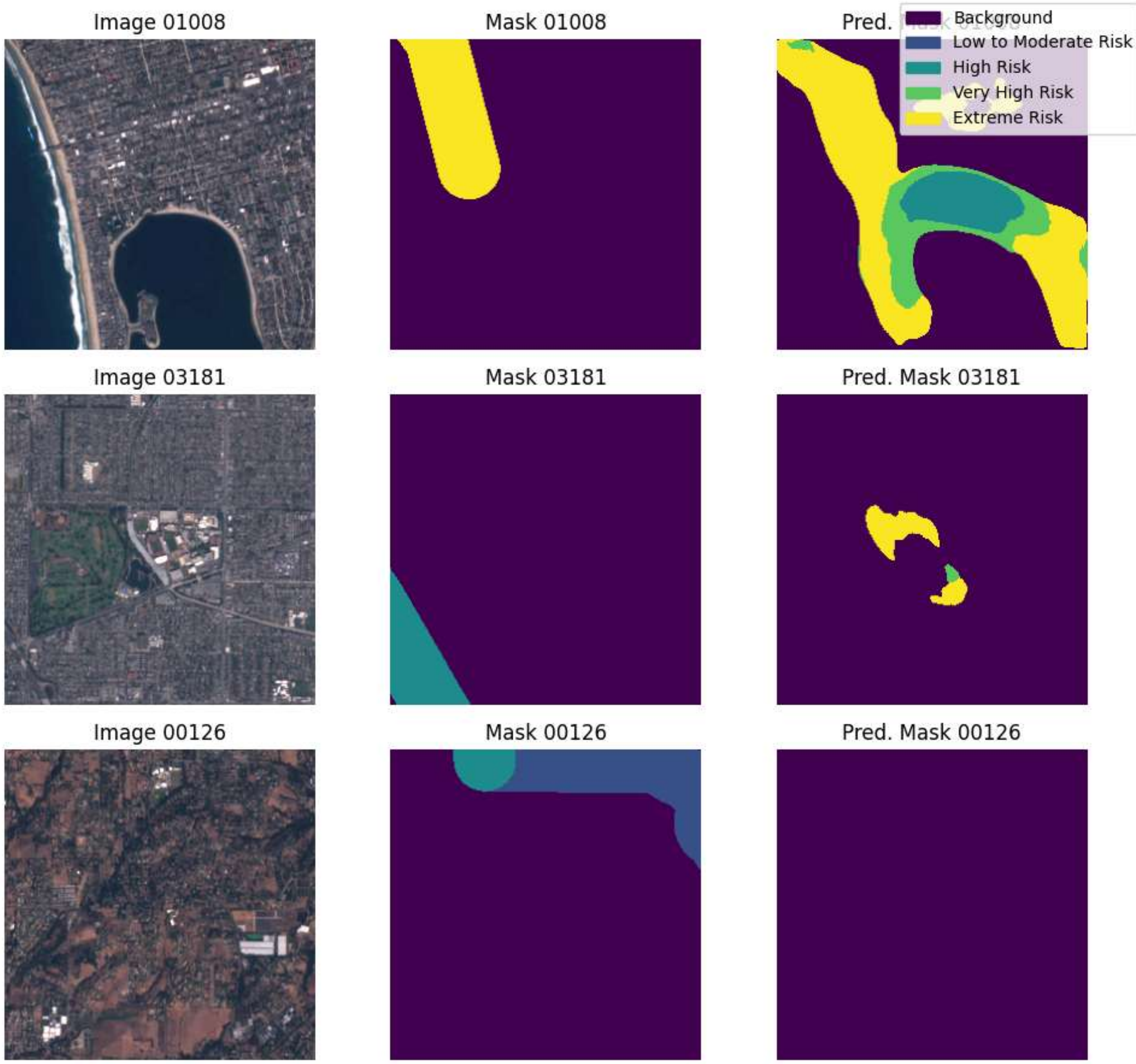
In [16]:

```
# Build the U-Net
# Specify the details of the optimizer
optimizer = optimizers.Adam(learning_rate=1e-4)  # Adam optimizer with a learning r

# Create the model
# NOTE: We have chosen to use some custom metrics iou_metric and dice_coeff. The de
metrics = [iou_metric, dice_coeff]  # List of custom metrics to use
custom_objects = {"iou_metric": iou_metric, "dice_coeff": dice_coeff}  # Custom obj

model = unet_model(
    the_weights = [0.27, 7.31/8, 2.73/8, 3.87/8, 2.13/8],
    input_shape=(256, 256, 3),  # Input shape of the images
    num_classes=num_classes,    # Number of classes
    num_blocks=4,               # Number of blocks in the U-Net model
    optimizer=optimizer,        # Optimizer to use
    metrics=metrics,            # Metrics to use
)

# Get a print out of the model structure.
model.summary()  # Print the summary of the model
```

```
Weights:  [0.27, 0.91375, 0.34125, 0.48375, 0.26625]
```

Model: "functional"

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 6s 343ms/step
Metrics based on test data:

Class Background Individual Metrics:
Precision: 0.8059022702539294
Recall: 0.9732340126622187
F1 Score: 0.8816991793425074

Class Low to Moderate Risk Individual Metrics:
Precision: 0.619627449399242
Recall: 0.2604922367618144
F1 Score: 0.3667867968209265

Class High Risk Individual Metrics:
Precision: 0.6883073157782281
Recall: 0.17433493061500263
F1 Score: 0.27820573045131525

Class Very High Risk Individual Metrics:
Precision: 0.20682672076946199
Recall: 0.1797988045856877
F1 Score: 0.19236803941781838

Class Extreme Risk Individual Metrics:
Precision: 0.4598831528783075
Recall: 0.18241987354407213
F1 Score: 0.2612219564350694
```



In [24]:

```
# Save the predicted masks
SAVE_PREDICTED_MASKS = True  # Flag to indicate whether to save the predicted masks
predicted_masks_dir = Path(f"{model_dir}/predicted_masks/")  # Directory to save th

if SAVE_PREDICTED_MASKS:
    save_predicted_masks(
        predicted_masks,            # Predicted masks to be saved
        predicted_masks_dir,        # Directory to save the predicted masks
        mask_names=names_test,      # Names of the test images corresponding to the
        file_ext="tif"              # File extension for the saved masks
    )
```

```
Data saved in models/predicted_masks.
```

In [25]:

```
# Save files associated with the predicted masks, or other files of interest.
save_associated_files(
    masks_dir,                # Directory containing the original masks
    predicted_masks_dir,      # Directory to save the associated files
    file_ext="tfw",           # File extension of the associated files to be saved
    file_names=names_test,    # Names of the test images corresponding to the associ
```